

Technika nawrotów (backtracking)

- dla problemów, których rozwiązania dają się przedstawić w postaci wektorów, niekoniecznie jednakowej długości;
- kolejny element (składowa) wektora-rozwiązania należy do ściśle określonego (dynamicznie) zbioru możliwych wartości tego elementu;
- często wykorzystuje rekurencję.

Schemat ogólny metody nawrotu

1. wygenerowawszy $i-1$ początkowych elementów wektora:

$$(x_1, \dots, x_{i-1})$$

ustal zbiór A_i możliwych wartości elementu i -tego, wybierz jedną z nich jako wartość x_i i kontynuuj dla wektora o 1 dłuższego.

2. Jeśli zbiór A_i jest pusty, lub wyczerpano wszystkie zawarte w nim możliwości, to wycofaj się do wektora $(x_1, \dots, x_{i-2})$ i kontynuuj analizę zbioru A_{i-1}

3. Po wygenerowaniu kompletnego wektora

$$(x_1, \dots, x_n)$$

wrót do wektora częściowego $(x_1, \dots, x_{n-1})$ i dobierz następny element ze zbioru A_n . Jeśli A_n pusty to cofaj się dalej, do wektora $(x_1, \dots, x_{n-2})$ i analizuj zbiór A_{n-1} , itd.

Pseudokod, wersja bez rekurencji

```
i := 1;   wektor := ();   koniec := FALSE ;
while NOT koniec do
    if (wektor kompletny) then
        zbadaj/wypisz wektor;
        if (wystarczy 1 rozwiązanie)
            koniec := TRUE;
        else
            // wykonaj BACKTRACK;
            if (i = 1) then
                koniec := TRUE;
            else
                i--;
    else
        ustal aktualny zbiór A(i);
        if (A(i) niepusty) then
            pobierz x ze zbioru A(i);
            dołącz x jako i-ty element wektora;
            i++;
        else
            wykonaj BACKTRACK;
end.
```

Przykład 1: Problem skoczka szachowego

Rozwiązanie metodą nawrotów.

Startując z lewego dolnego narożnika obejść szachownicę $N \times N$ ruchami skoczka szachowego, przechodząc przez każde pole dokładnie raz.

Numeracja możliwych kolejnych posunięć z danego położenia S:

	4		5	
3				6
		S		
2				7
	1		8	

Rozwiązanie: wektor $P[1..N^2]$ par $(x,y) \in [1, N] \times [1, N]$.
 $P[i] = (x,y)$ oznacza, że w i-tym ruchu skoczek został postawiony w polu (x,y) .

Tablicę pomocniczą (funkcja odwrotna do P):

Mark $[x, y] = i$

oznacza, że pozycja (x, y) znajduje się w wektorze P na miejscu i-tym (tzn. została zaliczona w i-tym ruchu).

Początkowo tablice P i Mark puste.

Main

```
// rekurencyjny algorytm dla skoczka szachowego
for (i=1,...,N, j=1,...,N) do
    Mark[i,j] := 0;
if (Move(1,1,1)) then
    output P;
else
    brak rozwiązania zaczynającego się od (1,1);
    // w tablicy P zostały śmieci
end.
```

```

Move (i, x, y)
// w i-tym ruchu skocz na pozycję (x,y),
// rekurencyjnie wykonaj ruchy i+1, i+2....
// zwróć 1 jeśli jest pełne rozwiązanie, 0 wpp.
// k, wynik - zmienne lokalne całkowite;

    P[i] := (x,y);
    Mark[x,y] := i;
    if (i = N*N) then return 1; // wektor kompletny

    // ustal następny ruch
    k := 1; // kierunek ruchu skoczka
    wynik := 0;
    while (wynik=0 && k<=8) do
        (c,r) := pozycja w kierunku k od (x,y);
        if ((c,r) na szachownicy & Mark[c,r]=0) then
            // można tam skoczyć
            wynik := Move(i+1,c,r);
            // powrót z rekurencji = BACKTRACK
        if (wynik=1) then return 1;
        k++; // następny możliwy kierunek
    end while;

    Mark[x,y] := 0;
    return 0;
end Move;

```

Uwagi:

- złożoność oczywiście wykładnicza;
- powyższą wersję rekurencyjną nietrudno zamienić na iteracyjną, bez dodatkowego stosu – rolę stosu rekurencji pełnić może tablica P;
- można zmienić algorytm tak, by generował wszystkie poprawne wektory.

Przykład 2: Kolorowanie (wierzchołkowe) grafu

Dane: graf nieskierowany $G=(V,E)$, liczba całkowita k ;

Wynik: przypisanie każdemu wierzchołkowi jednego z spośród k kolorów takie, że dowolne dwa wierzchołki sąsiednie mają przypisane różne kolory.

Metoda nawrotów wraz z odcinaniem rozwiązań (wektorów) częściowych, które nie mogą być rozwinięte do pełnych rozwiązań (niejawne "branch&bound"):

- krok 0: wybierz kolor dla wierzchołka 0;
- krok i : mając kolory wierzchołków $0..i-1$, wybierz kolor wierzchołka i ;
- jeśli krok i -ty niewykonalny, to wycofaj się do kroku $i-1$ i wybierz następny kolor dla wierzchołka $i-1$.

```

// wywołanie początkowe: Color(0)
// n = |V(G)|, 1..k = zakres kolorów,
// X = tablica kolorów

Color(v)          // przypisz kolor wierzchołkowi v
  for i = 1 to k do // próbujemy kolejne kolory
    X[v] := i;
    if Ok-Color(v) then
      if v = n-1 then
        output X[0..n-1]; exit;
        // pełne rozwiązanie, koniec
      else
        Color(v+1);
        // rekurencyjnie następny wierzchołek
      end Color;
    end Color;

Ok-Color(v)
// zakładając że X[0..v-1] to poprawne kolorowanie
// sprawdź czy X[0..v] jest poprawne
  for j = 0 to v-1 do
    if (j,v) ∈ E(G) oraz X[j] = X[v] then
      return false;
    end if;
  end for;
  return true;
end Ok-Color.

```

Branch & Bound

metoda rozgałęzień i ograniczeń

Znajdowanie rozwiązania dla problemu optymalizacyjnego.

Często: trudnego obliczeniowo.

- postać rozwiązania: wektor;
- funkcja kosztu $c(v)$, v - wektor rozwiązania;
- szukamy v takiego, że $c(v) = \min$;
- metoda nawrotów;

- dodatkowe usprawnienie:
 - funkcja kosztu wektora częściowego $b(v)$, ustalana heurystycznie, łatwo obliczalna;
 - własność funkcji $b()$ - monotoniczność:
 - dla każdego wektora rozwiązania v , dla każdego jego prefiksu u , zachodzi $c(v) \geq b(u)$
 - czyli:
rozwijając dany wektor częściowy u nie mamy szans znaleźć wektora pełnego o koszcie mniejszym niż $b(u)$.

Szkielet algorytmu:

```
znajdź jakikolwiek wektor rozwiązania v0;  
    // v0 może być sztuczny  
    // możemy też założyć że  $c(v0) = \infty$   
S := zbiór rozwiązań częściowych powstałych  
    z wyboru pierwszej składowej;  
while (S niepusty) do  
    wyjmij z S rozwiązanie u;  
    // są różne strategie optymalizacyjne wyboru u  
    if (u jest pełnym rozwiązaniem) then  
        if ( $c(u) < c(v0)$ ) then  
            v0 := u;  
        else pomiń u;  
    else  
        if ( $b(u) \geq c(v0)$ )  
            pomiń u; // odcięcie poddrzewa rozwiązań  
        else  
            wstaw do S wszystkie możliwe rozszerzenia u  
            o kolejną jedną składową;  
// S zawiera rozwiązania częściowe różnej długości
```

Przykład: komiwojażer

- graf pełny z wagami, $n=5$;
- startujemy z wierzchołka $v=0$;
- znaleźć taką permutację wierzchołków 1..4, aby droga komiwojażera (cykl Hamiltona) była najkrótsza.

Efektywność praktyczna mocno zależy od doboru funkcji $b()$.

Sprytne zdefiniowanie funkcji $b()$ pozwala omijać duże poddrzewa w drzewie nawrotów.

Można rozważać również różne strategie wyboru wektora u wyjmowanego z pojemnika: S może mieć swoją strukturę, np. stosu albo kolejki priorytetowej.